PM – Lucrarea 3

Această lucrare tratează șirurile de caractere și operațiile care se pot face în asamblare asupra caracterelor și a șirurilor de caractere.

3.1 Declararea unei variabile care poate să rețină un singur caracter

În limbajul de asamblare x86, o variabilă care reține la un moment dat un singur caracter se declară în același mod ca o variabilă cu lungimea de un byte, folosind directiva db – data byte. Pentru stocarea codului ASCII corespunzător unui caracter, sunt necesari 8 biți = un byte. Am putea spune că declarăm o variabilă de tip *char*, însă în asamblare nu avem tipuri, variabilele fiind declarate doar în funcție de lungime.

.data

ch1 db ’a’ ; *variabila ch1, pe un byte, reține caracterul ’a’, reprezentat prin codul ASCII al acestuia – 0x61 hexa (97 în zecimal)*

ch2 db ’1’

ch3 db 1 *; variabila ch3, pe un byte, are valoarea numerică inițială 1, în timp ce variabila ch2 reține caracterul ’1’, adică valoarea 0x31 hexa (49 în zecimal), corespunzătoare codului ASCII*

ch4 db ’?’ *; variabila ch4, pe un byte, are valoarea 0x3F (63 în zecimal), codul ASCII corespunzător caracterului ’?’*

În emulatorul emu8086, putem verifica codul ASCII în zecimal corespunzător fiecărui caracter prin apăsarea butonului „ascii codes” din meniul superior.

3.2 Afișarea unui caracter pe ecran, folosind întreruperea DOS INT 21h

Afișarea unui caracter pe ecran este similară cu afișarea unei valori numerice, atunci când ne referim la întreruperea INT 21h, cu serviciul de afișare:

MOV AH, 2

MOV DL, ’a’

INT 21H

Cele 3 linii de cod de mai sus vor avea ca efect afișarea pe ecran a caracterului ’a’. Același efect se poate obține scriind în DL valoarea codului ASCII corespunzător caracterului ’a’:

MOV AH, 2

MOV DL, 97 *; sau putem specifica valoarea în hexa: MOV DL, 61h*

INT 21H

3.3 Operații care pot fi folosite pe caractere

MOV DL, ’A’ *; în registrul DL se salvează caracterul ’A’*

ADD DL, 1 *; această adunare va avea de fapt ca efect stocarea în DL a caracterului imediat următor lui ’A’ în codul ASCII, adică ’B’*

MOV AH, 2

INT 21H *; pe ecran se va afișa litera ’B’*

Studiind tabelul cu codurile ASCII, observăm că diferența între codul ASCII al literei ’A’ și cel al literei ’a’ este de 32 în zecimal (0x20), această diferență păstrându-se pentru toate literele alfabetului. Acest concept poate fi folosit pentru crearea unei funcții care convertește literele mici în litere mari sau invers. Adunând 32 la o litera mare, obtinem litera mica corespunzătoare ei, iar dacă scădem 32 dintr-o literă mică, obținem litera mare corespunzătoare ei.

Codul de mai jos are ca efect transformarea unei litere mari citită de la tastatura în litera mică echivalentă

MOV AH, 1

INT 21H *; caracterul citit de la tastatură va fi stocat în registrul AL*

ADD AL, 32 *; dacă litera citită este mare, prin adunarea lui 32 se va transforma în literă mică*

MOV DL, AL *; întreruperea INT 21H afișează întotdeauna pe ecran conținutul registrului DL*

MOV AH, 2

INT 21H

3.4 Declararea și afișarea șirurilor de caractere

Declararea unui șir de caractere în limbajul de asamblare x86 se face în același mod ca pentru o variabilă care reține un singur caracter. Exemplu:

.data

msg db ‘Hello world!$’

În fapt, variabila pe un byte msg va reține doar primul caracter din șir, ‘H’, iar celelalte caractere vor fi salvate fiecare pe câte un byte la adresele succesive de memorie. Al doilea

caracter din șir ('e') poate fi accesat prin variabilă msg+1, al treilea caracter din șir ('l') poate fi accesat prin variabila msg+2, ș.a.m.d.

O altă modalitate de a accesa fiecare caracter al unui șir este folosind registrul SI – Source Index pe post de index în șir. Dacă inițializez SI cu 0, variabila msg[SI] va reține primul caracter din șirul msg. După incrementarea lui SI, variabila msg[SI] va reține al doilea caracter din șirul msg, ș.a.m.d.

Mai jos avem un exemplu de program simplu care afișează pe ecran șirul de caractere 'Hello world!':

```
.model small

.data

msg db 'Hello world!$' ; caracterul $ de la finalul șirului reprezintă un terminator de șir; acesta nu se va afișa în momentul afișării pe ecran a șirului de caractere

.code

main proc

        MOV AX, @data

        MOV DS, AX

        MOV AH, 9 ; în acest program, șirul de caractere se afișează folosind aceeași întrerupere INT 21H, însă cu serviciul 9 specific șirurilor

        LEA DX, msg ; prin instrucțiunea LEA – Load Effective Address, adresa de început a șirului de caractere msg este încărcată în registrul DX; aceasta este echivalentă cu instrucțiunea MOV DX, OFFSET msg, unde cuvântul cheie OFFSET înseamnă adresă de început

        INT 21H


        MOV AH, 4CH ; aceste instrucțiuni sunt folosite pentru ieșirea din DOS și întoarcerea la sistemul de operare

        INT 21H

        Ret

main endp

end
```

Pentru trecerea la linie nouă, se poate folosi combinația de caractere CR – Carriage Return și LF – Line Feed, care au codurile 10 și 13 în zecimal. Împreună, cele două caractere au efectul apăsării tastei „Enter”.

De exemplu, dacă în cadrul programului anterior, dorim ca primul cuvânt ‚Hello’ să se afișeze pe prima linie, iar al doilea cuvânt „world!” să fie afișat pe o linie nouă, putem insera caracterele speciale CR și LF direct în declararea șirului de caractere, astfel:

```
msg db ’Hello’,10,13,’world!$’
```

Întrebări

1. Care este valoarea codului ASCII, exprimat în hexa și în zecimal, pentru caracterul ’a’? Dar pentru caracterul ’A’?
2. Explicați în cuvinte principiul prin care pot să transform o litera mică oarecare în litera mare echivalentă. Cum pot să fac transformarea inversă, din literă mare în literă mică?
3. Care este caracterul special care se scrie la finalul unui șir de caractere, cu rol de terminator de șir?
4. Ce înseamnă caracterele speciale CR și LF? Care este rolul acestora?

Probleme

1. Scrieți un program în limbajul de asamblare x86 care citește un șir de caractere, format doar din litere mici, de la tastatură și afișează același șir scris doar cu litere mari.

2. Scrieți un program care citește, pe rând, două șiruri de caractere de la tastatură și le salvează în două variabile *msg1* și *msg2*. Finalul fiecărui șir de caractere va fi semnalat prin caracterul special $, care va fi salvat pe ultima poziție din fiecare șir. Programul va crea și va afișa pe ecran un nou șir obținut prin concatenarea celor două.

3. Scrieți un program care citește un șir de caractere de la tastatură și afișează un nou șir format doar din vocalele șirului inițial, în ordinea în care au fost găsite în șirul inițial.